QuipSwap Internal Audit

Repository	quip-swap-solidity
Repo URL	https://gitlab.com/quip.network/quip-swap-solidity
Branch	`v0`
Commit ID	86daa976a5d9d17dc93e92e13d6940be7dc64eab
Previous Audit	https://app.notion.com/p/QuipSwap-Audit-June-2026-375db8c6a08880d8b2d2e600efbcd628?source=copy_link

Audit Report Summary

- the main remaining concerns are misuse-sensitive cross-chain workflow assumptions, cross-chain reaction-gap sizing, accepted lock-poisoning grief, and trapped stray-asset recovery
- 1 informational issue: the repository still requires extra local bootstrap work before its test suite is reproducibly runnable
- the current Solidity code and test tree show deliberate hardening against the historically more serious swap failure classes, especially pooled-escrow drainage, cross-chain deadline mis-ordering, and reentrant recipient re-entry

Table Of Contents

1. Scope And System Overview
2. Misuse Weaknesses And Safety Assumptions
    a. Taker-first cross-chain commit after premature `lock` disclosure
    b. Cross-chain fairness depends on an unbounded reaction-gap width

    c. Mistaken observation of the wrong origin event
    d. Public lock poisoning as accepted grief
    e. Secret or swap confusion across sessions
    f. Recipient and reclaim-recipient misunderstanding
    g. Asset-behavior and stray-transfer assumptions
    h. Last-minute execution and chain-congestion timing risk
    i. Accepted grief conditions not understood by users
3. Operational And Repository Risks
    a. The repository still requires extra local bootstrap work before executable verification
    b. The deployment scripts are stale against the current `Omnibus` constructor
4. Testing Gaps
    a. Missing a runnable dependency tree in the checked-in repo state
    b. Missing explicit tests for accidental direct token transfers and recovery policy
    c. Missing broader invariant-style campaigns over swap lifecycle and mixed assets
    d. Missing direct tests for exposed receiver and interface behavior
5. Overall Assessment

Scope And System Overview

This review focused on the Solidity swap implementation in:

- `src/Omnibus.sol`
- `src/TermsLib.sol`
- `src/interfaces/IOmnibus.sol`
- the accompanying test tree under `test/`

The implemented swap model is:

- a party commits assets under terms that bind a chain, recipients, asset set, and a strictly ordered commit / claim / reclaim lifecycle
- the counterparty either commits the matching leg on the same chain or on another chain under an identical hashlock
- knowledge of the secret allows claim during the permitted window
- expiry of the claim flow allows reclaim back to the designated reclaim recipients

The key invariants enforced by the current design are:

- the two sides of a swap must have strictly interleaved lifecycle deadlines
- same-chain swaps must not advance to a fully committed state unless both sides were actually posted
- claim and reclaim must move a lock to `Spent` before any outbound transfer path can be re-entered
- fee withdrawals must not sweep escrowed native amounts that remain owed to swap participants

This pass included source review, test review, and executable validation. The repository did not start in a clean runnable state because the expected vendored dependency tree under `dependencies/` was missing, but after restoring the missing libraries locally I ran `forge test --no-match-path 'script/**'` successfully. The suite completed with 382 passing tests.

Interpretation note:

- the contract review is backed by an executed local test run
- the repository reproducibility story is still weaker than it should be, because a clean checkout did not reach that state without extra bootstrap work

The EasyCrypt `v06` line remains meaningful support for the intended protocol flow, but not for misuse traces that depend on premature `lock` disclosure, bad taker verification, or bypass of the guarded wallet workflow.

Misuse Weaknesses And Safety Assumptions

The implementation is strongest when used through the intended guided cross-chain flow. The following misuse-sensitive cases sit at the boundary between protocol assumptions, app behavior, and contract enforcement.

This should be read in the context of the design goal: a fully trustless cross-chain swap without bridge dependence, proof relay, or extra settlement transactions. That choice necessarily shifts more safety burden onto preimage secrecy, deadline selection, cross-chain observation, and correct user workflow.

1. Taker-first cross-chain commit after premature `lock` disclosure

Risk:

- the intended protocol flow assumes the taker learns `lock` only after observing the proposer's real commit on the origin chain
- if a malicious proposer, frontend, relayer, phishing flow, or other bad workflow discloses `lock` early, the taker can be induced to commit first on the taker chain
- because the pair is cross-chain, that first post becomes locally `Committed`, and the proposer can then claim the taker leg with the known secret without ever escrowing the opposite leg
- this same misuse cluster also covers:
  - treating possession of `lock` as sufficient evidence that proposer funding has happened
  - bypassing the guided QuipWallet flow through raw wallet calls, third-party frontends, custom scripts, or malicious relayers

Existing mitigations:

- `commit()` is restricted to factory-registered QuipWallet callers
- the guided QuipWallet flow can observe both chains and refuse taker commit until proposer escrow is confirmed.

🚀 **Possible future mitigations:**

- require chain-B-side evidence that proposer escrow already happened on chain A before a taker-side first post can become claimable
- redesign cross-chain commit so unilateral taker-first first post does not become locally claimable by the secret holder
- hard-block taker commit in the QuipWallet app until exact proposer escrow is observed on the origin chain
- present explicit warnings that possession of `lock` alone is not sufficient evidence that proposer funding has occurred
- treat the official cross-chain-aware QuipWallet app as a required safety boundary
- make dangerous raw call patterns harder to trigger in standard wallet UX
- provide stronger signing prompts that identify unsafe taker-side commits

2. Cross-chain fairness depends on an unbounded reaction-gap width

Risk:

- the implementation enforces strict ordering of the six lifecycle deadlines, but not minimum spacing
- it does not enforce any minimum width for the critical cross-chain reaction buffer after secret revelation
- a proposer can therefore choose schedules that are valid on-chain but operationally unfair in production conditions
- this same timing cluster also covers premature secret release, where the proposer reveals the secret too early, through the wrong channel, or to the wrong session and gives the counterparty settlement leverage earlier than intended

Existing mitigations:

- `validateTermPairing(...)` enforces strict interleaving and blocks the older dangerous mis-ordering class
- the current test tree already documents that the remaining risk is parameter sizing, not pure ordering

🚀 **Possible future mitigations:**

- reject deadline schedules below conservative chain-specific minimums in the QuipWallet app
- optionally enforce minimum reaction gaps on-chain
- add direct tests for valid-but-too-tight schedules so this boundary stays explicit
- bind UI workflows tightly around when and where the secret is revealed
- isolate secrets per swap and per session
- add operator or client checks that prevent early reveal outside the intended settlement step

3. Mistaken observation of the wrong origin event

Risk:

- a taker may be shown that some proposer commit happened, but on the wrong chain, wrong contract deployment, wrong environment, or under terms different from the exact swap they are about to fund
- the core failure is that the user or app binds taker readiness to the wrong origin-chain event

Existing mitigations:

- canonical terms hashing prevents later mismatch once both sides are committed
- chain IDs are embedded in terms and enforced on the local side

🚀 **Possible future mitigations:**

- present a strong canonical swap identity in the UI
- require exact origin-chain commit verification before enabling taker commit
- display chain, contract, asset, recipient, and amount checks together rather than as separate hints
- bind origin-chain verification to exact contract addresses and chain IDs
- surface environment labels clearly in the app
- require the UI to link taker commit readiness to the exact observed origin-chain transaction

4. Public lock poisoning as accepted grief

Risk:

- once the first commit transaction exposes the lock in the mempool, another registered QuipWallet can front-run with the same lock and different cheap terms
- the legitimate commit then fails with `TermsMismatch`

Existing mitigations:

- the attacker must escrow real assets under the attacker's own terms
- the attacker can only cause recoverable grief rather than direct counterparty theft under the currently documented behavior
- the test suite documents and pins this behavior in test/attack/LockPoisoning.t.sol

🚀 **Possible future mitigations:**

- if this is acceptable, document it plainly in user-facing and operator-facing materials
- if it is not acceptable, add a protocol-level reservation or authenticated first-commit binding rather than relying on retry with a fresh secret

5. Secret or swap confusion across sessions

Risk:

- poor secret handling, reuse, or bad swap labeling can cause users to associate the wrong off-chain negotiation with the wrong on-chain lock

Existing mitigations:

- the lock is derived from the secret and the terms are hashed canonically once committed

🚀 **Possible future mitigations:**

- ensure every swap uses unique freshly generated secret material
- present durable swap IDs in the UI
- prevent reusing lock material across distinct negotiations

6. Recipient and reclaim-recipient misunderstanding

This misuse case is about destination semantics, not swap validity. Settlement follows `recipient` for successful claim and `reclaimRecipient` for expiry-based recovery, so a user can approve a technically valid swap whose economic destination differs from what they intended.

Risk:

- users may assume funds always return to the signer or visible actor, even though payout flows follow `recipient` and `reclaimRecipient`

Existing mitigations:

- zero-address recipients are rejected
- terms hashing binds recipients once committed

🚀 **Possible future mitigations:**

- show recipients prominently before commit
- add explicit confirmation when recipient or reclaim recipient differs from the active wallet
- consider app-level safe defaults and warnings for unusual recipient layouts

7. Asset-behavior and stray-transfer assumptions

Risk:

- users may attempt swaps with fee-on-transfer or otherwise non-standard tokens and assume normal ERC-20 semantics
- the contract can also receive plain ETH, ERC20s, ERC721s, and ERC1155s outside tracked swap flow, and unsupported or accidental transfers may remain stuck permanently

Existing mitigations:

- `transferAssetsNoNative(...)` checks exact ERC20 and ERC1155 receipt and rejects under-delivery
- the test suite includes fee-on-transfer regression coverage
- `withdrawFees(...)` is intentionally limited to `accruedFees`, so it does not accidentally sweep live escrow
- the current tests already pin that stray native is not swept as fees

🚀 **Possible future mitigations:**

- classify supported and unsupported token behaviors clearly in the app and docs

- maintain a token allowlist or warning system for known non-standard assets
- add more adversarial token mocks for edge-case behaviors beyond under-delivery
- decide explicitly whether the intended policy is no rescue or a constrained rescue path
- if rescue is intentionally disallowed, document that clearly for operators and integrators
- add direct tests for accidental ERC20, ERC721, and ERC1155 sends so the behavior is pinned

8. Last-minute execution and chain-congestion timing risk

Risk:

- even when the chosen deadlines are valid and the user intends to follow the correct flow, acting too close to the edge of a commit, claim, or reclaim window can still fail because of mempool delay, congestion, inclusion latency, or other chain-local timing friction
- in cross-chain flows, that can turn a theoretically sufficient schedule into an operational miss if the user or app waits until the final safe-looking moments

Existing mitigations:

- the protocol already uses explicit deadline windows rather than instant one-shot actions
- the guided QuipWallet flow can monitor progress across both chains instead of relying on purely manual user timing

🚀 **Possible future mitigations:**

- apply conservative submission buffers in the QuipWallet app rather than treating the displayed deadline as the practical last moment

- warn users when a transaction is being attempted too close to the end of a critical window
- incorporate chain-specific confirmation and congestion assumptions into deadline recommendations
- add tests and operational runbooks that treat timing slack as a first-class safety margin

9. Accepted grief conditions not understood by users

Risk:

- users may misinterpret accepted grief cases such as lock poisoning, retriable contention, or failed near-deadline attempts as protocol failure and respond unsafely
- this is a meta UX and operations weakness: the technical condition may be recoverable, but the user's understanding of it may not be

Existing mitigations:

- the test suite documents lock poisoning as accepted, recoverable grief
- the protocol already separates reclaimable expiration states from completed settlement states on-chain

🚀 **Possible future mitigations:**

- explain retry behavior explicitly in user-facing flows
- auto-regenerate a fresh secret or lock after a poisoned attempt
- distinguish recoverable grief from fund loss in operator and user messaging
- surface clearer app states for retry, wait, reclaim, and terminal failure rather than relying on raw transaction outcomes

Operational And Repository Risks

1. The repository still requires extra local bootstrap work before executable verification

The checked-in Foundry configuration expects vendored packages under `dependencies/`, but the working tree did not initially contain that dependency tree. I was able to run the suite only after restoring the missing OpenZeppelin, `forge-std`, and Solady libraries locally and adding a minimal `ICreateX` interface stub so Forge could compile the deployment scripts.

Relevant references:

- foundry.toml
  - `libs = ["lib", "dependencies"]`
- foundry.toml
  - dependency declarations assume a vendored dependency workflow
- remappings.txt
  - imports resolve into `dependencies/...`

Impact:

- the suite is executable, but not from a clean checkout without extra local bootstrap work
- that weakens reproducibility for external reviewers and makes the repository look more self-contained than it currently is



Recommendation:

- restore the vendored dependency tree or make the repository bootstrap process reproducible from a clean checkout
- preserve a clean path for external reviewers to reach a passing `forge test` run without local patching

2. The deployment scripts are stale against the current `Omnibus` constructor

Both deployment scripts still encode `Omnibus` creation with only `OMNIBUS_OWNER`, but the current contract constructor requires both `initialOwner` and `quipFactory`.

Relevant references:

- src/Omnibus.sol
  - constructor requires `(address initialOwner, address quipFactory)`
- script/DeployOmnibus.s.sol
  - script encodes only `abi.encode(OMNIBUS_OWNER)`
- script/DeployTestnet.s.sol
  - script encodes only `abi.encode(OMNIBUS_OWNER)`

Impact:

- the repository's deployment path is stale against the audited contract surface
- a reviewer or operator could infer a higher deployment readiness than the current scripts actually provide
- if used as written, these scripts will not correctly initialize the current `Omnibus` constructor arguments

🚀 Recommendation:

- update both scripts to pass the current `quipFactory` argument explicitly
- add a deployment smoke test or script-level CI check so constructor drift is caught automatically

Testing Gaps

1. Missing a runnable dependency tree in the checked-in repo state

The immediate gap is practical rather than conceptual: the repository state under review does not compile because its vendored dependencies are absent. That prevents the current tests from serving as executed evidence for a clean checkout.

2. Missing explicit tests for accidental direct token transfers and recovery policy

The current test tree already covers stray native in the context of fee withdrawal, but it does not yet pin the intended behavior for:

- direct ERC20 transfers to the contract
- direct ERC721 safe transfers to the contract
- direct ERC1155 safe transfers to the contract
- direct ERC1155 batch transfers to the contract
- the expected operator policy if such assets are sent by mistake

These cases matter because the contract can already receive all of those asset types.

3. Missing broader invariant-style campaigns over swap lifecycle and mixed assets

The current test suite is materially stronger than a typical early-stage swap repository and already includes targeted regression coverage for the historically dangerous cases. The remaining gap is breadth, not absence.

The most useful next testing expansion would be:

- stateful or invariant-style campaigns over valid paired schedules
- broader mixed-asset permutations across native, ERC20, ERC721, and ERC1155 legs
- campaigns that assert lock-state monotonicity and one-action-per-window behavior across a wider range of traces
- campaigns that exercise valid-but-tight cross-chain deadline schedules and assert minimum acceptable reaction buffers

- explicit regressions for cross-chain commit-order safety, so taker-first commit can never again become a claimable theft state

4. Missing direct tests for exposed receiver and interface behavior

The contract exposes a small but real non-swap surface through `receive()`, ERC721 / ERC1155 receiver callbacks, and `supportsInterface(...)`. The current suite covers much of the surrounding asset behavior indirectly, but it does not yet pin these entrypoints directly on `Omnibus`.

The most useful direct additions would be:

- a plain-ETH `receive()` test that confirms direct native transfers are accepted and remain outside fee accounting
- direct tests for `supportsInterface(...)` over the declared supported and unsupported interface ids
- direct tests for `onERC721Received`, `onERC1155Received`, and `onERC1155BatchReceived`
- stronger event-argument assertions for `Claim` and `Reclaimed`, so emitted caller / recipient semantics stay explicit

Overall Assessment

This repository is materially stronger than earlier swap designs that failed on pooled escrow accounting, cross-lock fund confusion, or reentrant settlement ordering. The remaining weakness is that some cross-chain safety properties still depend on the guided wallet workflow and correct off-chain observation rather than purely on on-chain enforcement.

The main remaining weaknesses are:

- cross-chain fairness still depends on deadline sizing that is not enforced on-chain
- a residual lock-poisoning grief vector that still exists by design
- recoverability problems for assets sent outside tracked swap flows
- misuse-sensitive cross-chain flows still rely on the guarded QuipWallet workflow and correct origin-chain observation

- repository reproducibility still requires extra local bootstrap work before testing

Deployment note:

- the stale deployment scripts are important repository hygiene issues, but they are not the main economic-security issue in this report

Responsibility note:

- user-responsibility assumptions are acceptable only if the official QuipWallet flow actually enforces the required cross-chain verification before taker commit